Heapsort e Quicksort*

PCC104 – Projeto e Análise de Algoritmos

21 de abril de 2026

Conteúdo

1 Introdução **1**

2 Heapsort **1**
2.1 A estrutura de dados *heap* . . . 1
2.2 Max-Heapify: restaurando a propriedade de heap . . . 2
2.3 Construção do heap: Build-Max-Heap . . . 3
2.4 O algoritmo Heapsort . . . 4
2.5 Filas de prioridade . . . 5
2.5.1 Heap-Maximum . . . 5
2.5.2 Heap-Extract-Max . . . 6
2.5.3 Heap-Increase-Key . . . 6
2.5.4 Max-Heap-Insert . . . 6
2.5.5 Resumo das operações . . . 7
2.5.6 Aplicações . . . 7

3 Quicksort **7**
3.1 Descrição . . . 7
3.2 Partition: o núcleo do algoritmo . . . 8
3.3 Análise: pior caso . . . 8
3.4 Análise: melhor caso e caso balanceado . . . 8
3.5 Quicksort aleatorizado . . . 9
3.6 Análise do caso médio . . . 9

4 Comparação com os outros métodos **10**
4.1 Panorama . . . 10
4.2 Quando usar cada um? . . . 10
4.3 Diagrama das recorrências . . . 10

5 Exercícios sugeridos do CLRS **10**

1 Introdução

Nesta aula estudamos dois algoritmos de ordenação por comparação que, embora compartilhem o tempo assintótico $O(n \lg n)$ com o *merge sort*, apresentam propriedades bastante distintas:

- Heapsort: ordena *in place* e garante $O(n \lg n)$ no **pior caso**. A ideia central é uma estrutura de dados implícita — o *heap* — que mantém uma ordem parcial sobre os elementos.
- Quicksort: ordena *in place*, tem tempo $\Theta(n^2)$ no pior caso, mas seu **caso médio é** $O(n \lg n)$ com constantes muito pequenas. Na prática, é o mais rápido dos três para entradas grandes e aleatórias.

Ambos os algoritmos são comparativos: assumimos apenas que os elementos admitem uma relação de ordem total e que podemos testá-la com operações de comparação.

2 Heapsort

2.1 A estrutura de dados *heap*

Definição 2.1 (Heap binário como arranjo)**.** Um heap binário é um arranjo $A[1 \mathinner{..} n]$ que visualizamos como uma árvore binária quase completa, preenchida da esquerda para a direita. Para um nó de índice i, definimos:

$$\textsc{Parent}(i) = \lfloor i/2 \rfloor, \qquad \textsc{Left}(i) = 2i, \qquad \textsc{Right}(i) = 2i + 1.$$

Dois atributos acompanham o arranjo: `A.length`, o número de posições alocadas, e `A.heap-size`, o número de elementos válidos do heap (sempre `A.heap-size` $\leq$ `A.length`).

*Material baseado em (Cormen et al., 2009).

Definição 2.2 (Propriedade de *max-heap*)**.** Um arranjo A satisfaz a propriedade de max-heap se, para todo índice $i > 1$,

$$A[\textsc{Parent}(i)] \geq A[i].$$

Equivalentemente, o valor de um nó não excede o valor de seu pai.

Altura. Definimos a altura de um nó como o número de arestas no caminho descendente mais longo até uma folha; a altura do heap é a altura de sua raiz. Como o heap é uma árvore binária quase completa, temos:

Lema 2.1. *Um heap de n elementos tem altura $\lfloor \lg n \rfloor$.*

Demonstração. A árvore é quase completa, então todos os níveis $0, 1, \ldots, h-1$ estão cheios, totalizando $2^h - 1$ nós, e o nível h tem entre 1 e 2^h nós. Assim $2^h \leq n \leq 2^{h+1} - 1$, o que dá $h = \lfloor \lg n \rfloor$. □

Consequência direta: operações que percorrem um caminho da raiz a uma folha — ou de uma folha à raiz — custam $O(\lg n)$.

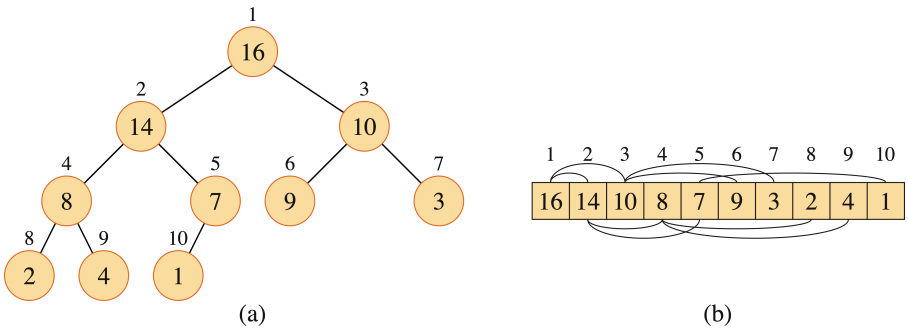

Figura 1: Estrutura de dados heap

2.2 Max-Heapify: restaurando a propriedade de heap

A operação Max-Heapify assume que as subárvores enraizadas em $\textsc{Left}(i)$ e $\textsc{Right}(i)$ já são max-heaps, mas $A[i]$ pode violar a propriedade localmente. O procedimento "desce" $A[i]$ pela árvore até restaurar a propriedade.

Algorithm 1 $\textsc{Max-Heapify}(A, i)$

```
ℓ ← Left(i)
r ← Right(i)
se ℓ ≤ A.heap-size e A[ℓ] > A[i] então
    largest ← ℓ
senão
    largest ← i
fim se
se r ≤ A.heap-size e A[r] > A[largest] então
    largest ← r
fim se
se largest ≠ i então
    troca A[i] ↔ A[largest]
    Max-Heapify(A, largest)
fim se
```

Proposição 2.2. *O tempo de execução de Max-Heapify sobre um nó de altura h é $O(h)$; em particular, $O(\lg n)$ para o heap inteiro.*

Demonstração. O trabalho local em cada chamada é $\Theta(1)$: comparações com os filhos e, possivelmente, uma troca. A recursão desce por uma única subárvore, de forma que

$$T(h) \leq T(h-1) + \Theta(1),$$

cuja solução é $T(h) = O(h)$. Como $h \leq \lfloor \lg n \rfloor$ (Lema 2.1), concluímos $T = O(\lg n)$. □

Observação 2.1. A recorrência que aparece no CLRS é $T(n) \leq T(2n/3) + \Theta(1)$, onde o fator $2/3$ corresponde ao tamanho máximo de uma subárvore filha em um heap quase completo (pior caso quando o último nível está pela metade). Pelo método mestre (caso 2), $T(n) = O(\lg n)$.

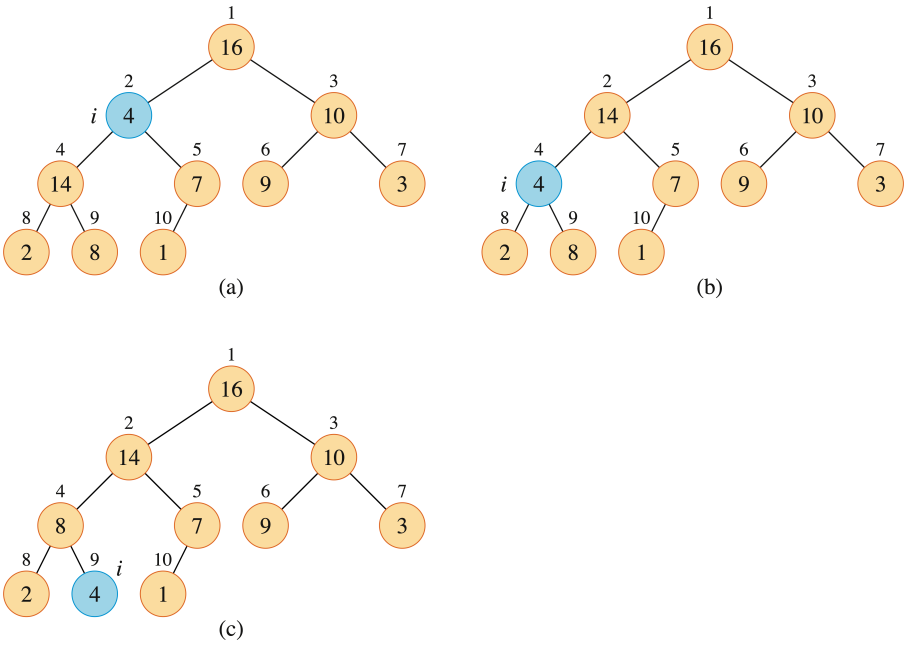

Figura 2: Execução Max-Heapify(A,2)

2.3 Construção do heap: Build-Max-Heap

Dado um arranjo arbitrário, construímos um max-heap chamando Max-Heapify nos nós internos, de baixo para cima. Nós nos índices $\lfloor n/2 \rfloor + 1, \ldots, n$ são folhas — cada um é, trivialmente, um max-heap unitário.

Algorithm 2 Build-Max-Heap(A)

```
A.heap-size ← A.length
para i ← ⌊A.length/2⌋ até 1 faça
    Max-Heapify(A, i)
fim para
```

Corretude (invariante de laço). Antes da primeira iteração, cada nó $\lfloor n/2 \rfloor + 1, \ldots, n$ é raiz de um max-heap (folhas). Antes de cada iteração com índice i, cada nó $i+1, i+2, \ldots, n$ é raiz de um max-heap. A chamada Max-Heapify(A, i) preserva essa propriedade e estende-a a i. No fim, $i = 0$ e todo nó é raiz de um max-heap, em particular $A[1]$.

Teorema 2.3. *Build-Max-Heap executa em tempo $O(n)$.*

Demonstração. A análise ingênua dá $O(n \lg n)$: são $\Theta(n)$ chamadas, cada uma $O(\lg n)$. Mas podemos apertar o limite observando que o custo de Max-Heapify em um nó depende da *altura* do nó, não de n.

Em um heap de n elementos, o número de nós de altura h é no máximo $\lceil n/2^{h+1} \rceil$. Logo, o custo total é

$$T(n) \;=\; \sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) \;=\; O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right).$$

Usando a identidade $\sum_{h=0}^{\infty} h/2^h = 2$, obtemos $T(n) = O(n)$. □

Ou seja: construir o heap é **linear**. É um resultado surpreendente à primeira vista, mas decorre do fato de que a grande maioria dos nós está próxima das folhas, onde Max-Heapify custa pouco.

A soma que aparece na análise de Build-Max-Heap é

$$\sum_{i=0}^{h} 2^i (h-i) = 2^{h+1} - h - 2.$$

Como $h = \lfloor \lg n \rfloor$, temos $2^h \le n < 2^{h+1}$, logo $2^{h+1} \le 2n$. Portanto

$$\sum_{i=0}^{h} 2^i (h-i) \;=\; 2^{h+1} - h - 2 \;\le\; 2n - \lfloor \lg n \rfloor - 2 \;=\; O(n).$$

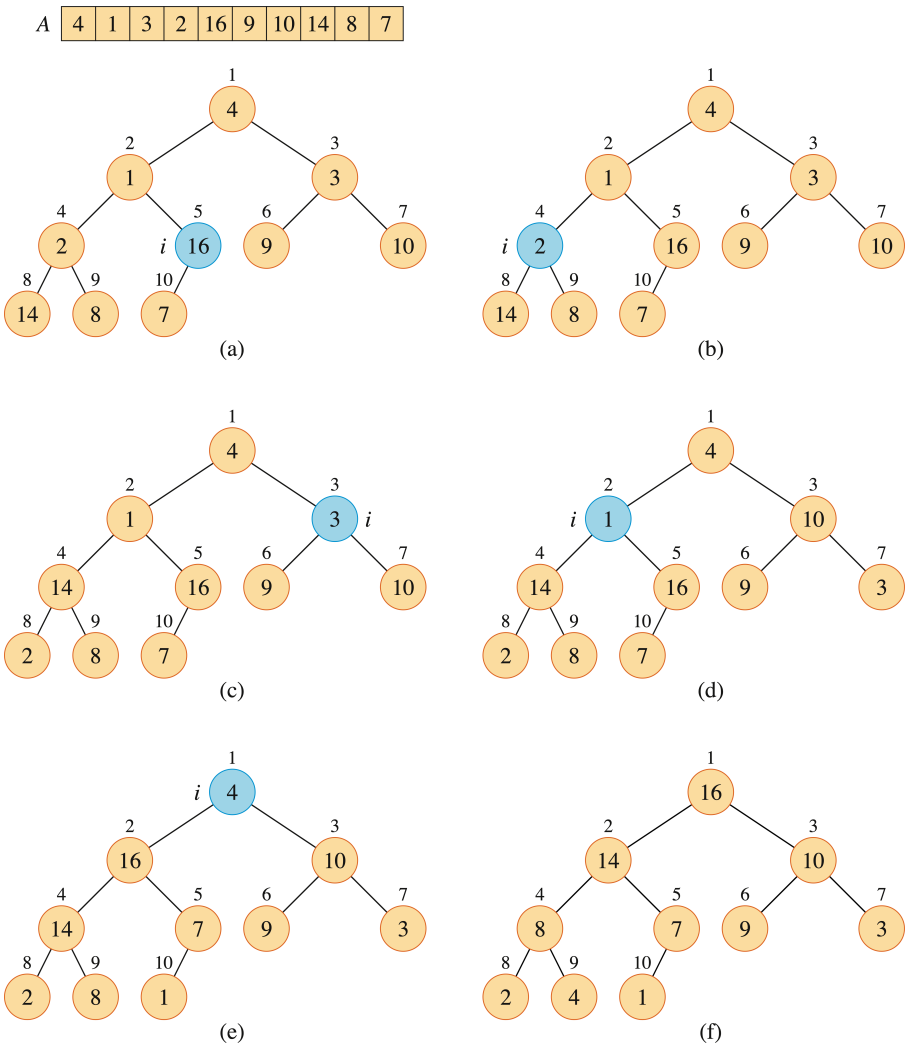

Figura 3: Execução Build-Max-Heap

2.4 O algoritmo Heapsort

A ideia é simples: o máximo do arranjo está em $A[1]$. Trocamos $A[1]$ com $A[n]$, reduzimos o heap em uma unidade e restauramos a propriedade chamando Max-Heapify. Repetimos até sobrar um único elemento.

Algorithm 3 Heapsort(A)

```
Build-Max-Heap(A)
para i ← A.length até 2 faça
    troca A[1] ↔ A[i]
    A.heap-size ← A.heap-size − 1
    Max-Heapify(A, 1)
fim para
```

Teorema 2.4. *Heapsort ordena um arranjo de n elementos em tempo $O(n \lg n)$ no pior caso.*

Demonstração. A chamada inicial a Build-Max-Heap custa $O(n)$ (Teorema 2.3). O laço executa $n-1$ iterações; cada iteração faz trabalho $\Theta(1)$ mais uma chamada a Max-Heapify na raiz, de custo $O(\lg n)$ (Proposição 2.2). Total:

$$T(n) = O(n) + (n-1) \cdot O(\lg n) = O(n \lg n). \qquad \square$$

Corretude. O invariante do laço é: no início de cada iteração, o subarranjo $A[i+1\,..\,n]$ contém os $n-i$ maiores elementos em ordem ordenada, e $A[1\,..\,i]$ é um max-heap. A troca coloca o maior de $A[1\,..\,i]$ em $A[i]$, e Max-Heapify restaura a propriedade sobre $A[1\,..\,i-1]$. No fim, $i = 1$ e todo o arranjo está ordenado.

Exemplo 2.1. Considere $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$. Após Build-Max-Heap, $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$. As iterações subsequentes vão "drenando" o máximo para o final do arranjo, produzindo a ordem crescente.

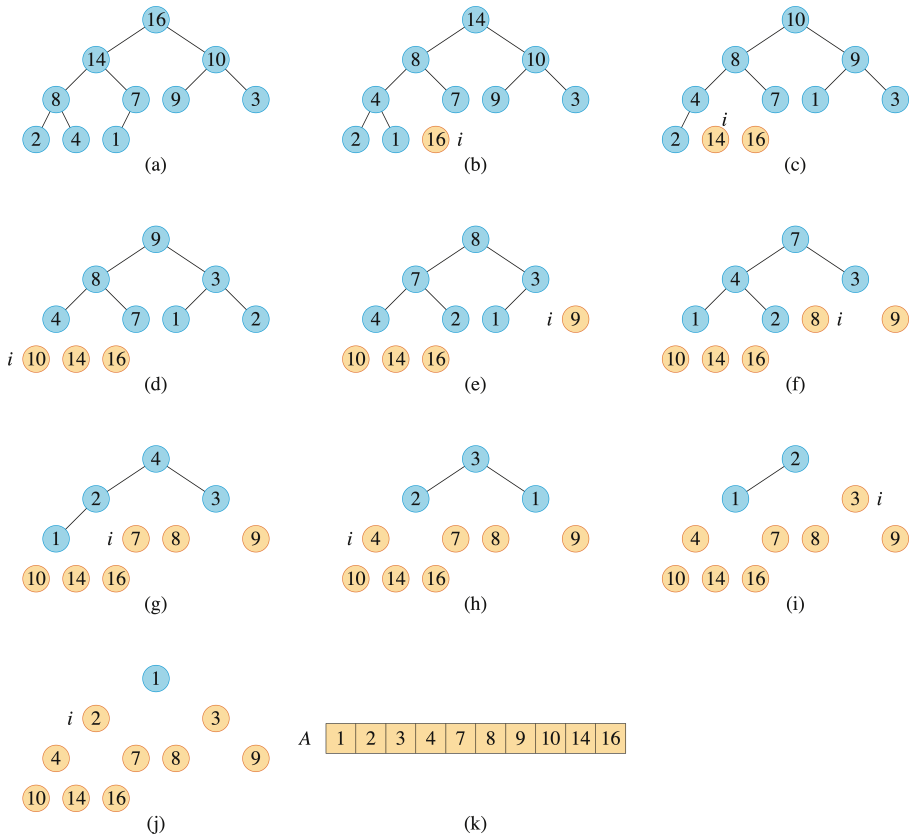

Figura 4: Execução Heap-Sort

2.5 Filas de prioridade

Uma das aplicações mais importantes do heap é a implementação de filas de prioridade, uma estrutura de dados abstrata extremamente útil em algoritmos de escalonamento de tarefas, simulações dirigidas por eventos, algoritmos de grafos (Dijkstra, Prim), e muitas outras aplicações.

Definição 2.3 (Fila de prioridade)**.** Uma fila de prioridade é uma estrutura de dados que mantém um conjunto S de elementos, cada um associado a uma **chave** (valor numérico). Há duas variantes:

- Fila de prioridade máxima: suporta operações em que o elemento de **maior** chave tem prioridade.
- Fila de prioridade mínima: análoga, com o elemento de **menor** chave.

Nesta seção tratamos da variante **máxima**, implementada sobre um max-heap. A variante mínima é obtida por simetria, usando um min-heap.

Operações. Uma fila de prioridade máxima suporta:

- Maximum(S): retorna o elemento de S com maior chave.
- Extract-Max(S): remove e retorna o elemento de S com maior chave.
- Increase-Key(S, x, k): aumenta a chave do elemento x para o novo valor k (que se assume $\geq$ chave atual).
- Insert(S, x): insere x em S.

Na prática, os elementos costumam ser *handles* para objetos externos (tarefas, eventos, vértices), e a chave é uma propriedade numérica (prioridade, tempo de chegada, distância). A fila de prioridade ordena logicamente esses elementos por chave, sem precisar de um *sort* global.

2.5.1 Heap-Maximum

Trivial: em um max-heap, o maior elemento está na raiz.

Algorithm 4 Heap-Maximum(A)

```
1: retorna A[1]
```

Tempo: $\Theta(1)$.

2.5.2 Heap-Extract-Max

Para extrair o máximo, devolvemos $A[1]$, mas precisamos manter a propriedade de heap. A ideia é colocar o último elemento na raiz e descer com Max-Heapify.

Algorithm 5 Heap-Extract-Max(A)

```
se A.heap-size < 1 então
    erro "heap underflow"
fim se
max ← A[1]
A[1] ← A[A.heap-size]
A.heap-size ← A.heap-size − 1
Max-Heapify(A, 1)
retorna max
```

Proposição 2.5. *Heap-Extract-Max executa em tempo $O(\lg n)$.*

Demonstração. Todo o trabalho fora da chamada a Max-Heapify é $\Theta(1)$. A chamada custa $O(\lg n)$ pela Proposição 2.2. □

Corretude. Antes da chamada a Max-Heapify, a raiz $A[1]$ pode violar a propriedade de max-heap, mas ambas as subárvores de $A[1]$ *continuam* sendo max-heaps — nenhuma operação anterior as modificou. Essa é exatamente a pré-condição de Max-Heapify, que restaura a propriedade em $O(\lg n)$.

2.5.3 Heap-Increase-Key

Aumentar a chave de um elemento pode violar a propriedade de max-heap apenas "para cima" — o novo valor pode exceder o valor do pai. A operação percorre o caminho até a raiz, trocando com o pai sempre que necessário.

Algorithm 6 Heap-Increase-Key(A, i, key)

```
se key < A[i] então
    erro "nova chave é menor que a atual"
fim se
A[i] ← key
enquanto i > 1 e A[Parent(i)] < A[i] faça
    troca A[i] ↔ A[Parent(i)]
    i ← Parent(i)
fim enquanto
```

Invariante do laço. No início de cada iteração, o arranjo $A[1\,..\,A.\text{heap-size}]$ satisfaz a propriedade de max-heap, **exceto possivelmente** pelo par $(A[i], A[\textsc{Parent}(i)])$, onde $A[i]$ pode ser maior que $A[\textsc{Parent}(i)]$.

Inicialização. Antes da primeira iteração, a única posição alterada foi $A[i]$. Como as subárvores de $A[i]$ continuam max-heaps (seus valores não mudaram) e todos os outros pares pai–filho também permanecem corretos, a única possível violação é entre $A[i]$ e seu pai.

Manutenção. Se $A[\textsc{Parent}(i)] < A[i]$, a troca corrige a violação local. Mas agora o valor de $A[i]$ (antigo $A[\textsc{Parent}(i)]$) pode ser menor que o outro filho do pai? **Não**: o outro filho já era $\leq A[\textsc{Parent}(i)]$ antes da troca, e $A[\textsc{Parent}(i)]$ recebeu o valor antigo de $A[i]$, que é maior ainda. Logo, a única possível violação nova está entre o novo $A[\textsc{Parent}(i)]$ e *seu* pai — e é exatamente o que o novo valor de i ($\textsc{Parent}$(i)) aponta.

Término. Ou $i = 1$ (chegamos na raiz, nada acima para violar), ou $A[\textsc{Parent}(i)] \geq A[i]$ (a propriedade é satisfeita localmente, e por indução em toda a árvore).

Proposição 2.6. *Heap-Increase-Key executa em tempo $O(\lg n)$.*

Demonstração. Cada iteração do laço sobe um nível na árvore. Como a altura é $\lfloor \lg n \rfloor$ (Lema 2.1), o laço executa no máximo $\lfloor \lg n \rfloor$ iterações, cada uma em tempo $\Theta(1)$. □

2.5.4 Max-Heap-Insert

Inserir um novo elemento com chave k é fácil de implementar reutilizando Heap-Increase-Key: colocamos $-\infty$ na nova posição (última folha) e então "aumentamos" essa chave para k.

Algorithm 7 MAX-HEAP-INSERT(A, *key*)

```
A.heap-size ← A.heap-size + 1
A[A.heap-size] ← −∞
HEAP-INCREASE-KEY(A, A.heap-size, key)
```

Proposição 2.7. *MAX-HEAP-INSERT executa em tempo $O(\lg n)$.*

Demonstração. As duas primeiras linhas custam $\Theta(1)$; a chamada a HEAP-INCREASE-KEY custa $O(\lg n)$. □

Truque pedagógico: usar $-\infty$ como chave inicial garante que a propriedade de max-heap vale trivialmente antes da chamada a HEAP-INCREASE-KEY (qualquer valor é maior), e assim a operação de subida está sempre bem definida.

Exemplo 2.2. Sobre o heap $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, executar MAX-HEAP-INSERT$(A, 15)$:

1. Coloca $-\infty$ em $A[11]$: $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1, -\infty]$.
2. HEAP-INCREASE-KEY$(A, 11, 15)$: aumenta para 15.
3. Sobe: $A[11] = 15 > A[5] = 7 \Rightarrow$ troca.
4. Sobe: $A[5] = 15 > A[2] = 14 \Rightarrow$ troca.
5. Sobe: $A[2] = 15 < A[1] = 16 \Rightarrow$ para.

Resultado: $[16, 15, 10, 8, 14, 9, 3, 2, 4, 1, 7]$.

2.5.5 Resumo das operações

Operação	Tempo	Mecanismo
HEAP-MAXIMUM	$\Theta(1)$	Lê raiz
HEAP-EXTRACT-MAX	$O(\lg n)$	Troca e desce (MAX-HEAPIFY)
HEAP-INCREASE-KEY	$O(\lg n)$	Sobe trocando com pai
MAX-HEAP-INSERT	$O(\lg n)$	Inserção + INCREASE-KEY

Operação relacionada: HEAP-DELETE(A, i), que remove o elemento $A[i]$, também pode ser feita em $O(\lg n)$ (Exercício 6.5-8 do CLRS). A ideia é substituir $A[i]$ pelo último elemento e chamar MAX-HEAPIFY ou HEAP-INCREASE-KEY, dependendo da comparação.

2.5.6 Aplicações

Filas de prioridade são ubíquas. Alguns usos clássicos:

- Escalonamento em sistemas operacionais: processos são extraídos em ordem decrescente de prioridade.
- Simulação dirigida por eventos: eventos futuros são armazenados em uma fila de prioridade mínima pela chave "tempo de ocorrência"; o simulador sempre extrai o próximo evento.
- Algoritmo de Dijkstra (caminhos mínimos em grafos): usa uma fila de prioridade mínima de vértices, com chave = distância estimada à origem, e executa DECREASE-KEY ao relaxar arestas.
- Algoritmo de Prim (árvore geradora mínima): estrutura idêntica à de Dijkstra, adaptada para peso mínimo ligando a árvore.
- Codificação de Huffman: usa uma fila de prioridade mínima para construir iterativamente a árvore de códigos a partir das frequências.

3 Quicksort

3.1 Descrição

O quicksort segue o paradigma divisão e conquista, com a divisão feita por **particionamento em torno de um pivô**.

- **Divide:** escolhe um pivô $x \in A[p\,..\,r]$ e particiona o arranjo em dois subarranjos (possivelmente vazios) $A[p\,..\,q-1]$ e $A[q+1\,..\,r]$ tais que todo elemento do primeiro é $\leq x$ e todo elemento do segundo é $\geq x$. Posição q é o índice final do pivô.
- **Conquista:** ordena recursivamente $A[p\,..\,q-1]$ e $A[q+1\,..\,r]$.
- **Combina:** nada a fazer — o arranjo já está ordenado.

Algorithm 8 QUICKSORT(A, p, r)

```
se p < r então
    q ← PARTITION(A, p, r)
    QUICKSORT(A, p, q − 1)
    QUICKSORT(A, q + 1, r)
fim se
```

3.2 Partition: o núcleo do algoritmo

Usamos o esquema de Lomuto, como no CLRS: escolhe-se $A[r]$ como pivô e mantém-se um índice i que separa elementos $\leq x$ dos $> x$.

Algorithm 9 Partition(A, p, r)

```
x ← A[r]
i ← p − 1
para j ← p até r − 1 faça
    se A[j] ≤ x então
        i ← i + 1
        troca A[i] ↔ A[j]
    fim se
fim para
troca A[i + 1] ↔ A[r]
retorna i + 1
```

Invariante de laço. No início de cada iteração do `for`:

(i) Se $p \leq k \leq i$, então $A[k] \leq x$.
(ii) Se $i + 1 \leq k \leq j - 1$, então $A[k] > x$.
(iii) $A[r] = x$.

Inicialização. Antes da primeira iteração, $i = p - 1$ e $j = p$: as faixas (i) e (ii) estão vazias; (iii) vale pela atribuição.

Manutenção. Considere a iteração com valor j. Se $A[j] > x$, incrementamos j e $(i, j-1)$ passa a conter $A[j]$, preservando (ii). Se $A[j] \leq x$, incrementamos i, trocamos $A[i] \leftrightarrow A[j]$ e incrementamos j: o novo $A[i] \leq x$ (preservando (i)) e o elemento que estava em $A[i]$ — que era $> x$ — move-se para $A[j - 1]$ (preservando (ii)).

Término. $j = r$. Pelo invariante, $A[p\,..\,i] \leq x$ e $A[i + 1\,..\,r - 1] > x$. A troca final coloca o pivô na posição $i + 1$, que é portanto o índice devolvido.

Proposição 3.1. *Partition executa em tempo $\Theta(n)$, onde $n = r - p + 1$.*

Demonstração. O laço **for** itera $n - 1$ vezes; cada iteração faz trabalho $\Theta(1)$. □

3.3 Análise: pior caso

Teorema 3.2. *O tempo de pior caso do quicksort sobre n elementos é $\Theta(n^2)$.*

O pior caso ocorre quando o particionamento é maximamente desbalanceado em cada chamada — por exemplo, quando o arranjo já está ordenado (ou em ordem inversa), de modo que o pivô é sempre o maior (ou menor) elemento. A partição produz então um lado com $n - 1$ elementos e outro vazio.

Demonstração. Seja $T(n)$ o tempo de pior caso. Existe particionamento maximamente desbalanceado (mostrado acima), produzindo a recorrência

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n),$$

cuja solução, por expansão, é

$$T(n) = \sum_{k=1}^{n} \Theta(k) = \Theta(n^2).$$

Para o limite superior, mostramos por indução — usando o método da substituição — que $T(n) = O(n^2)$. Suponha $T(k) \leq ck^2$ para todo $k < n$. Como o particionamento produz dois subproblemas de tamanhos q e $n - q - 1$ com $0 \leq q \leq n - 1$, e como $q^2 + (n - q - 1)^2$ é maximizado nos extremos $q \in \{0, n - 1\}$, temos

$$T(n) \leq \max_{0 \leq q \leq n-1} \{cq^2 + c(n - q - 1)^2\} + \Theta(n) \leq c(n - 1)^2 + \Theta(n) \leq cn^2,$$

para c suficientemente grande. Logo $T(n) = \Theta(n^2)$. □

3.4 Análise: melhor caso e caso balanceado

Melhor caso. Se o particionamento é perfeitamente balanceado em cada chamada, $T(n) = 2\,T(n/2) + \Theta(n)$, e pelo método mestre (caso 2), $T(n) = \Theta(n \lg n)$.

Particionamento "razoável". O resultado mais interessante do CLRS é que mesmo partições *consistentemente desbalanceadas* — mas em proporção constante — ainda dão $O(n \lg n)$:

Proposição 3.3. *Se todo particionamento divide o arranjo em frações α e $1-\alpha$ com $0 < \alpha \leq 1/2$ constante, então $T(n) = O(n \lg n)$.*

Ideia. A recorrência é $T(n) = T(\alpha n) + T((1-\alpha)n) + \Theta(n)$. A árvore de recursão tem profundidade $\Theta(\lg n)$ (dominada pelo ramo mais longo, de tamanho $((1-\alpha))^k n$, que atinge 1 em $\log_{1/(1-\alpha)} n = \Theta(\lg n)$ passos) e cada nível custa $O(n)$ (lembre: o custo em um nível é a soma dos tamanhos, e essa soma não aumenta com a profundidade). Logo $T(n) = O(n \lg n)$. □

Intuição. Particionamentos bons e ruins se alternam de forma natural em entradas aleatórias. Um nível "ruim" seguido de um "bom" dá o mesmo efeito de dois níveis "razoáveis". Isso motiva a análise probabilística.

3.5 Quicksort aleatorizado

Para evitar que entradas adversárias produzam o pior caso sistematicamente, escolhemos o pivô aleatoriamente:

Algorithm 10 RANDOMIZED-PARTITION(A, p, r)

```
i ← RANDOM(p, r)
troca A[r] ↔ A[i]
retorna PARTITION(A, p, r)
```

O RANDOMIZED-QUICKSORT simplesmente chama RANDOMIZED-PARTITION no lugar de PARTITION. O pivô passa a ser uniformemente distribuído entre os elementos do subarranjo.

3.6 Análise do caso médio

Teorema 3.4. *O tempo esperado do RANDOMIZED-QUICKSORT sobre uma entrada de n elementos distintos é $O(n \lg n)$.*

A prova segue a técnica de contagem de comparações, central no CLRS.

Observação-chave. O tempo total é dominado pelo número de comparações realizadas em todas as chamadas a PARTITION. Mais precisamente, se X é o número total de comparações, então o tempo é $O(n + X)$.

Indicadores. Renomeie os elementos de A como $z_1 < z_2 < \cdots < z_n$ (estatísticas de ordem). Para $i < j$, seja

$$X_{ij} = \mathbb{I}\{z_i \text{ é comparado a } z_j\}.$$

Então $X = \sum_{i<j} X_{ij}$. Pela linearidade,

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr[z_i \text{ é comparado a } z_j].$$

Lema 3.5. $\Pr[z_i$ *é comparado a* $z_j] = \dfrac{2}{j-i+1}$.

Demonstração. z_i e z_j só podem ser comparados **em uma chamada a PARTITION cujo intervalo contenha ambos**. Considere o conjunto $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$, de cardinalidade $j - i + 1$. O primeiro elemento de Z_{ij} a ser escolhido como pivô separa o caso em dois cenários:

- Se o pivô é z_i ou z_j, então z_i e z_j **são** comparados (pivô contra não-pivô).
- Se o pivô é algum z_k com $i < k < j$, então z_i e z_j vão para subarranjos distintos — e **nunca mais** serão comparados.

Como cada elemento de Z_{ij} tem igual probabilidade de ser o primeiro pivô escolhido em Z_{ij} (propriedade da escolha uniforme), a probabilidade de o pivô inicial ser z_i ou z_j é $\dfrac{2}{j-i+1}$. □

Prova do Teorema 3.4. Usando o Lema 3.5,

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k} \leq \sum_{i=1}^{n-1} 2H_n = O(n \lg n),$$

onde $H_n = \sum_{k=1}^{n} 1/k = \Theta(\lg n)$ é o n-ésimo número harmônico. Logo $\mathbb{E}[X] = O(n \lg n)$, e o tempo total esperado é $O(n + n \lg n) = O(n \lg n)$. □

Observação 3.1. A análise não depende de nenhuma hipótese sobre a entrada: a aleatoriedade está no algoritmo, não nos dados. Este é o grande ganho conceitual do quicksort aleatorizado.

4 Comparação com os outros métodos

4.1 Panorama

A tabela abaixo resume as características principais dos quatro métodos vistos no curso:

Algoritmo	Melhor caso	Pior caso	Caso médio	In place?	Estável?
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	Sim	Sim
Merge sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	Não	Sim
Heapsort	$\Theta(n \lg n)$[1]	$\Theta(n \lg n)$	$\Theta(n \lg n)$	Sim	Não
Quicksort (aleat.)	$\Theta(n \lg n)$	$\Theta(n^2)$	$\Theta(n \lg n)$ (esp.)	Sim	Não

4.2 Quando usar cada um?

Insertion sort. Apesar de $\Theta(n^2)$, é extraordinariamente rápido para n pequeno (constante < 10) — por isso é usado como *caso base* em implementações híbridas de merge/quicksort. Também é o melhor para arranjos *quase ordenados*: $\Theta(n + d)$, onde d é o número de inversões.

Merge sort. Escolha natural quando (i) estabilidade é essencial, (ii) os dados não cabem em memória (*external sort*), ou (iii) a entrada está em listas encadeadas. Custo: precisa de $\Theta(n)$ memória adicional.

Heapsort. Garantia rígida de $O(n \lg n)$ *in place* — interessante em sistemas de tempo real onde pior caso importa. Na prática, porém, costuma ser $\sim 2\times$ mais lento que o quicksort em entradas aleatórias, por causa do padrão de acesso à memória (pouca localidade de cache).

Quicksort aleatorizado. Padrão de fato para dados em memória. Constantes pequenas, excelente localidade (o particionamento varre o arranjo linearmente), e pior caso raríssimo em prática. É o algoritmo por trás de `qsort` da libc, `std::sort` do C++ (híbrido com heapsort — *introsort*), e das implementações de `sort()` em Java e Python (Timsort, que é um híbrido com merge sort).

4.3 Diagrama das recorrências

Merge sort
$T(n) = 2T(n/2) + \Theta(n)$
$\Rightarrow \Theta(n \lg n)$

Heapsort
$\textsc{Build} + (n-1) \cdot O(\lg n)$
$\Rightarrow O(n \lg n)$

Quicksort pior
$T(n) = T(n-1) + \Theta(n)$
$\Rightarrow \Theta(n^2)$

Quicksort médio
$\mathbb{E}[X] = O(n \lg n)$
via comparações

5 Exercícios sugeridos do CLRS

Para fixação, recomendam-se os seguintes exercícios do Cormen et al. (2009):

- **Heapsort:** 6.1-1, 6.1-2, 6.2-5 (iterativo), 6.3-3, 6.4-2, 6.5-7.
- **Quicksort:** 7.1-1, 7.1-2, 7.2-2, 7.2-3, 7.3-1, 7.4-2, 7.4-5 (combinação com insertion sort).

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[1] Rigorosamente, $\Omega(n)$ se todos os elementos são iguais; $\Theta(n \lg n)$ no caso genérico.